School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory #8: Working with Related Data

Objectives:

- To read related data with multi-table JOINs, subqueries and views.
- To run those queries inside controllers and return the result over HTTP.
- To design nested resource routes such as /students/:id/courses.
- To shape flat JOIN rows into nested JSON in JavaScript.
- To create and remove relationships safely, with validation and foreign keys.
- To show and edit a many-to-many relationship from a front-end page.

1. Recap and Goal

Up to now each route has read or written a single table — students, or courses, on their own. But real data is connected: a student takes many courses, and a course has many students. This lab works across tables inside your application: reading related data with JOINs, returning it as nested JSON, and creating and removing the links between records. The structure from Lab 6 — a shared pool, routers and controllers — stays exactly the same; we simply add routes that span more than one table. We continue with the students, courses and enrolments schema.

2. The Relationships in Our Schema

students and courses have a many-to-many relationship: each student can take many courses, and each course can have many students. We record those links in the enrolments table — the junction table you designed in Lab 2. Each enrolment row holds a student_id and a course_id (two foreign keys) and optionally a grade. So the question 'which courses does this student take?' is answered by joining enrolments to courses and filtering by the student.

3. Joining Tables in a Controller

A controller can run the same INNER JOIN you wrote by hand in Lab 3 and return the result as JSON. Backticks let the SQL string span several lines for readability. Mount the route as usual; only the query changes.

```
exports.enrolments = async (req, res) => {
  const [rows] = await pool.query(`
    SELECT s.name, c.title, e.grade
    FROM enrolments e
    JOIN students s ON e.student_id = s.id
    JOIN courses  c ON e.course_id  = c.id`);
  res.json(rows);
};
```



4. Multi-Table Joins

You can join more than two tables in one query, and combine a JOIN with GROUP BY to summarise related data. A LEFT JOIN is useful when you want to keep rows that have no match — for example, students who are enrolled in no courses still appear, with a count of zero. The query below lists each student with the number of courses they take.

```
SELECT s.name, COUNT(e.course_id) AS course_count
FROM students s
LEFT JOIN enrolments e ON e.student_id = s.id
GROUP BY s.id
ORDER BY course_count DESC;
```

5. Subqueries

A subquery is a query nested inside another. The inner SELECT runs first and the outer query uses its result — a tidy way to answer 'which rows relate (or do not relate) to others'. The common forms use IN and NOT IN against a set returned by the subquery. The first example finds courses with no enrolments; the second finds students taking more than two courses.

```
-- courses nobody is enrolled in
SELECT title FROM courses
WHERE id NOT IN (SELECT course_id FROM enrolments);

-- students who take more than two courses
SELECT name FROM students
WHERE id IN (
  SELECT student_id FROM enrolments
  GROUP BY student_id HAVING COUNT(*) > 2);
```

6. Views: a Saved Query

A view is a named query you can SELECT from as if it were a table. Define a join once as a view, then reuse it everywhere — in Workbench or in a controller — without repeating the SQL. A view stores the query, not the data, so it always reflects the current rows. Views keep complex joins in one place and make your controllers shorter.

```
CREATE VIEW enrolment_details AS
  SELECT s.name AS student, c.title AS course, e.grade
  FROM enrolments e
  JOIN students s ON e.student_id = s.id
  JOIN courses  c ON e.course_id  = c.id;

-- then query it like a table:
SELECT * FROM enrolment_details WHERE student = 'Suda';
```

7. Nested Resources: a Student's Courses

When one thing belongs to another, we express it in the URL: /students/:id/courses reads as 'the courses of student :id'. This is clearer than a flat /courses?student=:id and mirrors how the data is related. The same nested path serves reading, creating and removing a student's enrolments. The id from the URL goes into the query through a ? placeholder, exactly as before.

```
// routes/students.routes.js
router.get('/:id/courses', c.courses);

// controllers/students.controller.js
exports.courses = async (req, res) => {
  const [rows] = await pool.execute(`
    SELECT c.id, c.title, e.grade
    FROM enrolments e
    JOIN courses c ON e.course_id = c.id
    WHERE e.student_id = ?`, [req.params.id]);
  res.json(rows);
};
```

8. Shaping Related Data as Nested JSON

A JOIN returns flat rows — one per enrolment, repeating the student's name on every row. Often the client wants the data nested instead: a single student object with a courses array inside. The database gives you rows; JavaScript shapes them, which is where Lab 1's objects and arrays pay off. Walk the rows once, keyed by student id in a Map, collecting each student's courses into a list, then return the assembled objects.

```
// rows: { student_id, name, title }   (from a JOIN)
const byStudent = new Map();
for (const r of rows) {
  if (!byStudent.has(r.student_id))
    byStudent.set(r.student_id, { id: r.student_id, name: r.name, courses: []
});
  byStudent.get(r.student_id).courses.push(r.title);
}
res.json([...byStudent.values()]);
```

9. Creating a Relationship: Enrolling a Student

Enrolling a student in a course is simply inserting a row into the junction table that links the two ids. The student id comes from the URL and the course id from the request body; both go in through ? placeholders. Reply 201 with the new enrolment id.

```
// routes:  router.post('/:id/courses', c.enrol);
exports.enrol = async (req, res) => {
  const { course_id, grade } = req.body;
  if (!course_id)
    return res.status(400).json({ error: 'course_id is required' });
  const [r] = await pool.execute(
    'INSERT INTO enrolments (student_id, course_id, grade) VALUES (?, ?, ?)',
    [req.params.id, course_id, grade ?? null]);
  res.status(201).json({ id: r.insertId });
};
```

10. Validation and Referential Integrity

A link is only valid if both sides exist — you cannot enrol in a course that is not there. Foreign keys enforce this: MySQL rejects an enrolment whose course_id has no matching course, raising an error your controller should catch and turn into a clear 400 (or 409) response rather than a crash. Decide your rule for duplicates too: a UNIQUE key on (student_id, course_id) stops the same enrolment being

created twice. Validation in the controller and constraints in the database work together to keep the data honest.

11. Removing a Relationship: Un-enrolling

Removing a link deletes the junction row — the student and the course themselves stay. Two ids from the URL identify exactly one enrolment. Delete it, and if nothing matched, reply 404; otherwise 204 No Content.

```
// routes:  router.delete('/:id/courses/:courseId', c.unenrol);
exports.unenrol = async (req, res) => {
  const [r] = await pool.execute(
    'DELETE FROM enrolments WHERE student_id = ? AND course_id = ?',
    [req.params.id, req.params.courseId]);
  if (r.affectedRows === 0)
    return res.status(404).json({ error: 'enrolment not found' });
  res.status(204).end();
};
```

Because of referential integrity you cannot delete a student who still has enrolments — that would orphan those rows. Remove from the 'many' side first: delete the enrolments, then the student, exactly as in Lab 3.

12. A Front-End for Relationships

Now make the relationship visible and editable on a page. Fetch the nested JSON for one student and render their courses, each with a remove button. A select of all courses plus an Enrol button POSTs to the nested route to add one. As always, re-load the student after every change so the page matches the database.

```
async function show(id) {
  const s = await (await fetch(`/students/${id}`)).json();
  name.textContent = s.name;
  courseList.innerHTML = s.courses.map(c =>
    `<li>${c.title}
       <button data-course="${c.id}">remove</button></li>`
  ).join('');
}

enrolForm.onsubmit = async (e) => {
  e.preventDefault();
  await fetch(`/students/${id}/courses`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ course_id: courseSelect.value })
  });
  show(id);
};
```

13. Good Habits

Express relationships in the URL so a route reads like the data: /students/:id/courses is 'this student's courses'. Let foreign keys guard your links, and catch the resulting error to return a clear status rather than a crash. Shape data into nested JSON in JavaScript when the client needs it, keeping the SQL itself simple. Create and remove links through the junction table, and never delete a parent row that still has children.

14. Lab Exercises (complete during class)

Exercise 1: Advanced reads. In Workbench, list each course with the number of students enrolled (JOIN + GROUP BY); use a subquery to list the students not enrolled in any course; then CREATE VIEW enrolment_details joining students, enrolments and courses, and SELECT from it for one student.

Exercise 2: Read related data. Add GET /students/:id/courses to your students router and controller: join enrolments to courses, filter by the student id with a ? placeholder, and return each course title and grade. Test it for a student who has enrolments.

Exercise 3: Shape nested JSON. Add GET /students/:id that returns the student plus a courses array, assembling the nested object in JavaScript from the JOIN rows. A student with no enrolments should get an empty array, not an error.

Exercise 4: Create and remove links. Add POST /students/:id/courses that enrols the student in a course from the body (validate course_id is present), and DELETE /students/:id/courses/:courseId that un-enrols. Return 404 when the enrolment to remove does not exist.

Exercise 5: Relationships on the page. Build a page that shows a chosen student and the courses they take, with a select and Enrol button to add a course and a Remove button on each course. Re-load the student after every change.

15. Where We're Going Next

You can now read, create and remove relationships across tables and edit them from a page. But some actions touch several rows at once and must all succeed or all fail together, for example, recording a payment and the enrollment it pays for. In Lab 9 we wrap such multi-step operations in a transaction, so a change is all-or-nothing and your data never ends up half-updated.